



UNIVERSITÀ
DEGLI STUDI
FIRENZE

Scuola di Scienze Matematiche, Fisiche e Naturali
Laurea Magistrale in Data Science, Calcolo Scientifico &
Intelligenza Artificiale

Master Thesis

GRAPH NEURAL NETWORKS FOR MONEY
LAUNDERING DETECTION

MARTINO PAPA

Advisor: Dott. Daniele Castellana

Academic Year 2025-2026

ABSTRACT

CONTENTS

LIST OF FIGURES

"Citation"
— *citation's author*

INTRODUCTION

Money laundering (ML) is the criminal process of disguising or concealing the origin, identity, and ownership of illegally acquired funds. Its detection has become a foundational pillar of modern economic security. The rapid digitization of financial services has opened new possibilities for fraudulent activities. Undetected ML leads to massive economic losses for individuals and institutions, undermines the public trust in the financial system and frequently serves to finance criminal organizations. The scale of this phenomenon is staggering: the United Nations Office on Drugs and Crime (UNODC) estimates that between 2% and 5% of global GDP is laundered annually, with less than 1% of these illicit financial flows being successfully detected [? ?]. According to the Canadian Anti Fraud Centre (CAFC), 704 million dollars were lost because of frauds in 2025 only, confirming the raising trend of the previous years [?]. Consequently, developing robust mechanisms to identify and stop it is critically important to preserving both market integrity and social stability.

The goal of this dissertation is to evaluate the effectiveness of GNN-based models on detecting Money Laundering patterns. We will focus on both precision and explainability proposing two different tasks; a binary classification to label transactions as licit or illicit, and a multiclass classification to detect the specific type of laundering strategy employed.

1.1 TRADITIONAL AML METHODS

Anti Money Laundering is a critical component of the global financial system. Financial institutions ought to implement robust mechanisms to comply with regulatory standards and protect the integrity of the financial system [?].

Rule-based models have long been employed to detect suspicious transactions. These models consist of a set of predefined rules that are used to flag transactions that deviate from typical financial behavior. The

criteria are typically based on regulatory requirements, historical data, and known patterns of illicit behavior. For example we could decide to flag transactions that exceed a certain amount, that rapidly transfer funds between accounts or that involve a high-risk jurisdiction.

The main advantage of this approach is its transparency and interpretability. Furthermore they are computationally inexpensive, easy to implement and they can operate in real-time [?]. Unfortunately there are also several limitations. They tend to produce a high amount of false positives, which leads to more work from human operators that will have to deal with the flagged transactions. They could be easily exploited by criminals who are aware of the rules and can tailor their illicit activities to avoid detection. This will be further discussed in Section ??.

Due to its tendency of producing a huge amount of false positives and the huge amount of data that nowadays is produced by financial institutions, the rule-based approach is becoming less and less effective.

At the same time big datasets allow the usage of data-driven solutions that can learn directly from historical data. First attempts in this direction involved standard machine learning approaches such as logistic regression and support vector machines [?]. Those models improved the state of the art but they still struggled to detect human brain-armed patterns due to their very limited parameter capacity [?].

For this reason the research field shifted toward deep learning solutions which offer a much wider and complex hypothesis space, allowing to capture high order relationships between data, at the cost of a much worse interpretability. Nowadays the challenge is to combine high performance together with explainability to allow the usage of effective models in the highly regulated financial field, a necessity increasingly emphasized by both financial institutions and supervisory authorities [?]. We will discuss this further in Section ??

1.2 THE GROWING CHALLENGE

Detecting fraud is becoming increasingly difficult. The sheer volume and velocity of digital transactions make manual oversight impossible and heavily strain classical machine learning architectures. Furthermore, the rise of decentralized finance (DeFi), Web3, and cryptocurrencies has introduced a paradigm of pseudo-anonymity that complicates traditional tracing.

On top of that, criminals are constantly developing more and more sophisticated methods to avoid being caught. As Cuèllar (2003) noted,

"raising the marginal cost of perpetrating criminal activity can itself conceivably have counterintuitive effects. To the extent that criminals engaged in crimes for profit are acting rationally, then raising the cost of crime can discourage smaller operations and clear the market allowing more sophisticated criminal organizations to innovate and take more control from less organized criminals" [?]. Hence, to prevent inadvertently incentivizing sophisticated criminal networks, detection mechanisms must be updated continuously. The framework proposed in this study leverages deep learning models that support online updates. This ensures that the detection algorithm remains aligned with the most recent transaction patterns.

1.3 GNN FOR FRAUD DETECTION

The necessity of capturing complex interactions between accounts and transactions led to the idea of modeling the financial feed as a graph, where accounts are nodes and transactions are edges [?].

Definition 1.1 (Financial Graph). *A financial graph is a directed graph $G = (V, E)$, where V represents a set of financial entities (e.g. bank accounts or individuals), and E represents a set of transactions between these entities. Each edge $e \in E$ is directed from the sender to the receiver and may be associated with features.*

GNNs rely on message passing to aggregate information from neighbors and learn representations of the nodes that will be given as input to a classifier or regressor to predict the probability that an account (Node-Level detection), a transaction (Edge-Level detection) or a sub-graph (Graph-Level detection) is fraudulent. These methods allow to capture complex relationships, learn features automatically, handle dynamic graphs, and integrate multimodal data.

Formally the message passing process at the k -th layer in a GNN is defined as follows:

$$\begin{aligned} m_i^{(k)} &= \text{AGGREGATE} \left\{ \text{MSG} \left(h_j^{(k-1)}, e_{ji} \right) : j \in \mathcal{N}(i) \right\} \\ h_i^{(k)} &= \text{UPDATE} \left(h_i^{(k-1)}, m_i^{(k)}, x_i \right) \end{aligned} \tag{1.1}$$

where $\mathcal{N}(i)$ denotes the set of neighboring nodes of node i , e_{ji} is the feature vector of the edge from node j to node i , MSG (e.g. MLP) is the function that merges informations from the neighbor and the connecting edge and x_i is the feature vector of node i . Sometimes edge features

are not used during the message passing process, in such cases $m_i^{(k)}$ is defined as:

$$m_i^{(k)} = \text{AGGREGATE} \left\{ h_j^{(k-1)} : j \in \mathcal{N}(i) \right\} \quad (1.2)$$

After K iterations of message passing, the resultant node embeddings $H^{(K)}$ are then used to execute the designated task, in our case the detection of fraudulent accounts [?].

There are different types of GNNs; defined by how the UPDATE and AGGREGATE functions are implemented. Further details on those will be provided in Chapter ??.

1.4 STRUCTURE OF THE DISSERTATION

BACKGROUND

2.1 FINANCIAL CRIME AND MONEY LAUNDERING

Understanding how money laundering works is fundamental to guarantee both an effective and explainable detection of suspicious transaction. This section offers a theoretical introduction to money laundering providing a mathematical formulation of the problem together with a summary of common strategies employed by criminals to avoid detection.

2.1.1 *The Limitations of the Lonely Scammer*

Individual scammers typically rely on simple and intuitive techniques to introduce illicit funds into the legitimate economy. One of the most prevalent methods is known as “smurfing”. In simple terms, smurfing involves breaking down a large sum of illicit money into many smaller, inconspicuous chunks to deposit them without raising suspicion.

More formally, individual scammers rely on smurfing to avoid Anti-Money Laundering (AML) detection mechanisms, which usually flag transactions above a specific threshold $\tilde{x}$. By splitting their initial illicit funds x into smaller transactions x_n , such that each transaction satisfies $x_n \leq \tilde{x}$, they minimize the risk of detection [?]. The mathematical expectation of profit π_s for a scammer engaging in smurfing can be defined as:

$$\pi_s = x(1 - v) \tag{2.1}$$

where x represents the total initial illicit funds, and $v \in (0, 1)$ is the friction cost or commission proportion associated with the smurfing process (e.g., the fee paid to mules).

While smurfing is effective for small amounts, its scalability is severely limited. When attempting to launder much larger volumes of funds, an

individual scammer operating from a single account faces a significantly higher probability of detection $(1 - p)$, where p is the probability of successfully avoiding detection. If caught, the scammer is subjected to a severe monetary penalty T , which includes both the total confiscation of the funds and associated judicial fines [?]. Therefore, operating as a “lonely scammer” becomes mathematically and practically unviable for large-scale operations due to the massive expected cost of $(1 - p)T$.

2.1.2 *Fraud Rings*

In regard to overcome those issues, criminals have improved their techniques by employing highly sophisticated obfuscation tactics. Instead of acting alone, they form complex “fraud rings” to distribute illicit funds across vast networks of temporary accounts. To further break the traceable link between sender and receiver, illicit actors frequently use cyclic transfers and cryptocurrency **mixers** services that pool together funds from multiple users, mix them, and redistribute them at random intervals and amounts. These advanced tactics create dense, highly camouflaged transaction topologies that easily evade threshold-based rules and simple heuristic detection methods.

To achieve their goal they utilize a strategy of transaction layering, often characterized by a topological depth L , and network diversification to obscure the money trail [? ?]. By spreading the funds through a complex topology, the fraud ring successfully increases its non-detection probability p_L , which now depends explicitly on the layering depth L . The expected profit of a fraud ring utilizing L layers can be mathematically expressed as:

$$E[\pi_L] = p_L x - (1 - p_L)T - c(L) \quad (2.2)$$

where $E[\pi_L]$ is the expected profit, p_L is the enhanced probability of evading detection when L layers are used, x is the total illicit funds, T is the total penalty if caught, and $c(L)$ represents the monotonic cost function associated with establishing and maintaining the L transaction layers. Because the fraud ring dramatically increases the probability of evading detection ($p_L \rightarrow 1$ as L increases), the expected payoff remains highly profitable even when factoring in the complex layering costs $c(L)$ [?].

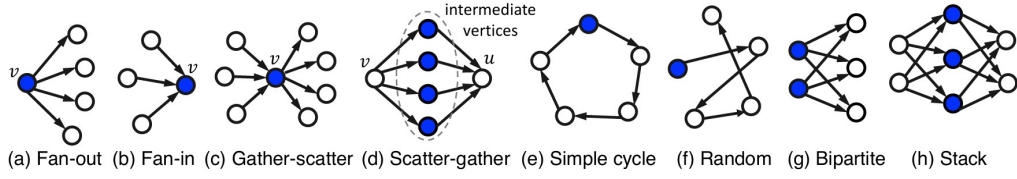


Figure 1: Money Laundering patterns

2.1.3 Money Laundering patterns

Understanding the structural patterns that criminality assumes is fundamental to develop effective detection models and to employ strategies to enhance the explainability of them. According to the literature, there are 8 different patterns that are often used in ML [? ?]. Which are:

- *fan-out*, shown in Figure ?? a, characterized by outgoing edges from a node v to more than 2 adjacent nodes;
- *fan-in*, Figure ?? b, v is connected to more than 2 nodes by incoming edges;
- *gather-scatter*, Figure ?? c, is the combination of a fan-in and a fan-out pattern on the same node v ;
- *scatter-gather*, Figure ?? d, also composed by a fan-in and a fan-out pattern but on two different nodes, sharing the same intermediate neighbors;
- *simple-cycle*, Figure ?? e, characterized by a sequence of transactions that starts and ends at the same node;
- *random*, Figure ?? f, similar to the cycle but founds do not return to the original account;
- *bipartite*, Figure ?? g, moves founds from a set of original nodes to a set of target nodes, with no transactions between accounts of the same set;
- *stack*, Figure ?? h, extends the bipartite pattern by adding a second bipartite layer.

It's important to mention that in some cases some patterns can be found in both illicit and licit transactions [?]. Specifically, *fan-in* and *fan-out* are patterns that are also frequent in legitimate, every-day transactions,

such as salary payments or transfers between family members or friends. That being said, the detection of such patterns remains a fundamental component of any money laundering detection models, albeit insufficient on its own. Furthermore, a model that correctly classifies those patterns provides a much more interpretable result than one that simply performs a binary classification between licit and illicit transactions. In Section ?? we will see why explainability is of fundamental importance in the financial field.

2.2 ETHICS, PRIVACY AND REGULATIONS

In this section we will begin by introducing a brief overview of the regulations that currently govern the field of AML. Subsequently a discussion on the main issues that the use of ML models may introduce will be provided, together with suggested solutions to address them. A special focus will be reserved to the issues of explainability and privacy. To conclude we will consider the use of Synthetic Datasets as a possible solution to these issues, analyzing their limits and advantages in the context of AML.

2.2.1 AML Global Regulations

Effective ML detection requires a combination of domestic laws, regulatory oversight, and international cooperation. Since the end of the 20th century the United States and the European Union as key jurisdictions, together with the FAFT ¹ as the global leading standard-setter have been developing and elaborate framework for AML [?].

In 1989 the G-7 nations established an inter-governmental task force to develop and promote international guidelines in regard to fight money laundering, the FATF. Nowadays FAFT still holds a central role in the AML field. It has issued 40 recommendations for national laws and international cooperation [?]. These principles have been adopted by over than 200 jurisdictions and they fall into 5 main areas [?]:

- Risk Based Approach (RBA): jurisdictions must identify and understand the financial crime risks that they face, allocating the right amount of resources to mitigate them;

¹ The Financial Action Task Force (FATF) is an intergovernmental policymaking body established in 1989 by the G7

- Legal Framework: members nations must adopt laws that criminalize money laundering; furthermore authorities must be given the power to freeze and confiscate funds when needed;
- Preventive Measures:
 - Customer Due Diligence (CDD): financial institutions must verify the identity of their customers;
 - Ultimate Beneficial Ownership (UBO): company's owners must be verified, this addresses the issue of criminals hiding behind shell institutions;
 - Record Keeping: financial institutions must maintain their transactions records for at least 5 years and file Suspicious Transactions Reports (STRs) to Financial Intelligence Units (FIUs).
- Supervision and Transparency: regulators must actively supervise financial institutions; members nations must immediately enforce UN financial sanctions when required to.
- International cooperation: members must quickly share information and provide mutual legal assistance in international investigations.

In regard to those guidelines FAFT keeps a *black-list* and a *grey-list* of jurisdictions that are not compliant with its standards [?].

2.2.2 Ethical and practical issues

When considering real-world datasets, models that detect financial crime are trained on sensitive data that needs to be protected. To avoid the leakage of sensitive information, before feeding ML models, it is necessary to apply privacy-preserving techniques. The most common practice is to apply differential privacy, which adds noise to the dataset to protect individual data points. That noise can be quantified using the privacy budget ϵ , which determines the trade-off between privacy and utility. All this comes at the cost of reducing the accuracy of the model.

Another significant issue to consider is explainability. Regulators require banks to explain their decisions, and this requirement extends to the ML models they use. Deep Learning models, albeit generally more precise and powerful than traditional ML models, are often *black-boxes*, meaning their decision process is very difficult to understand for a human being.

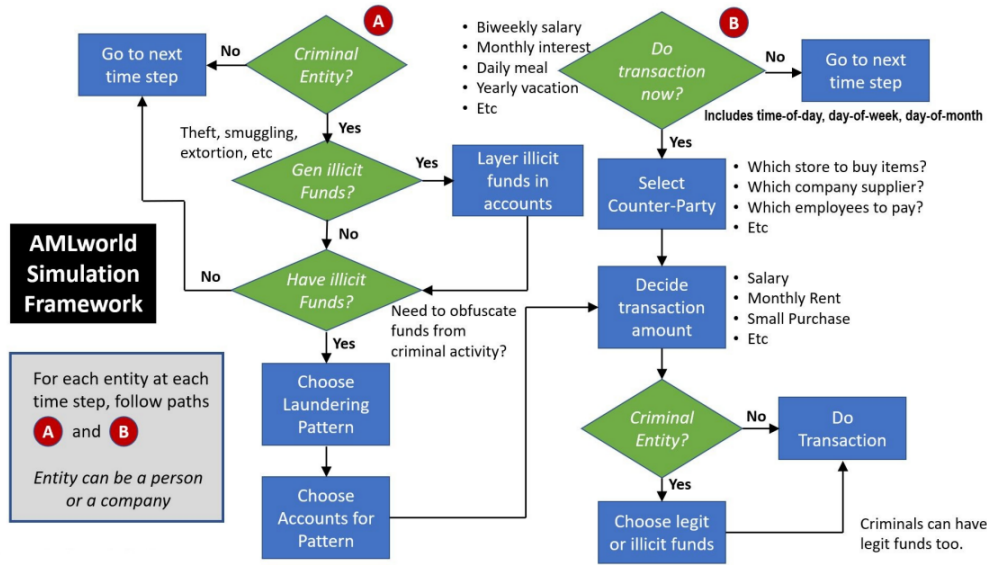


Figure 2: Overview of AMLworld simulation [?]

Real datasets are also often limited by the fact that the informations are not shared accross jurisdictions, making it difficult to detect frauds that cross borders.

To tackle all those problematics researchers at IBM developed a simulator for generating realistic financial transactions, called AMLWorld (2023) [?]. This simulator solves the privacy problem, simulates multi-jurisdictional transactions and it even tries to takle the explainability issue by providing as lable not only a boolean value for the presence of a fraud, but also the ML pattern that characterized it. This wouldn't be possible in real life, but models trained on this synthetic dataset could be able to generalize the detection of patterns even in real world scenarios, highly enhancing interpretability. Figure ?? provides an overview on how AMLWorld generates realistic financial transactions and patterns of financial crime.

The possible laundering patterns are the one described in Section ??; the model assumes the existence of 9 different surces of criminal activities: extortion, loan sharking, gambling, prostitution, kidnapping, robbery, embezzlement, drugs, and smuggling. The amount of money and the frequency of these criminal enterprises vary by both activity category and acting entity [?]. The funds are then inserted in the financial system alongside the legit ones.

2.3 TASK

In this dissertation we consider the problem of Edge-Level ML detection. This means, considering the Graph $G = (V, E)$ as in Definition ??, trying to learn a function f that, for each transaction $e \in E$, predicts its class. In particular we will consider two different settings. A binary classification where each edge is labeled as fraudulent or legit and a multi-class classification where, in case of illicit transaction, we also predict the type of pattern, amongst the one described in Section ??, it's involved in.

Formally we seek a function f that minimizes the error $L(f)$ over a distribution $\mathcal{D}$ of Graphs.

$$\operatorname{argmin}_{f \in \mathcal{H}} \{L(f) \doteq \mathbb{E}_{(x,y) \sim \mathcal{D}}[l(f(x), y)]\} \quad (2.3)$$

Practically there's no way to know $\mathcal{D}$. Instead we will consider a dataset $G_D = (V_D, E_D)$, where both nodes and edges are equipped with feature vectors and we will seek a function f that minimizes $L(f)$ over this dataset computing the empirical risk:

$$\operatorname{argmin}_{f \in \mathcal{H}} \left\{ \hat{L}(f) \doteq \frac{1}{|E_D|} \sum_{e \in E_D} l(f(x_e), y_e) \right\} \quad (2.4)$$

This is just the theoretical overview of the problem. In Chapter ?? we will provide more insights on how to define the hypothesis space $\mathcal{H}$ and the loss function l .

As mentioned in Section ??, the strict privacy regulations that governs the financial world, makes it hard to find real-world datasets. In this dissertation, for the reasons explained in Section ??, we will use the synthetic dataset *IBM AMLSim*. This is perfect for our scope, since it provides edge labels for both binary and multiclass classification.

2.3.1 Dataset description

The IBM AMLSim dataset comes in 6 different versions.

- HI-Small
- HI-Medium
- HI-Large
- LO-Small

	Small		Medium		Large	
Statistic	HI	LI	HI	LI	HI	LI
# of Days Spanned	10	10	16	16	97	97
# of Bank Accounts	515K	705K	2077K	2028K	2116K	2064K
# of Transactions	5M	7M	32M	31M	180M	176M
# of Laundering Transactions	3.6K	4.0K	35K	16K	223K	100K
Laundering Rate (1 per N Trans)	981	1942	905	1948	807	1750

Figure 3: IBM AMLSim datasets statistics [?]

- LO-Medium
- LO-Large

Where HI (High) and LO (Low) indicate the frequency of illicit transactions and the suffixes Small, Medium and Large indicate the size of the dataset. Figure ?? shows the differences between the 6 datasets.

In this work we will use the HI-Small version. Bigger datasets would of course guarantee a better generalization and accuracy but, for hardware and time constraint, it would be impossible to train the models on such graphs. That being said, we believe that the obtained results are good enough to understand how the models would perform in a real world scenario.

The HI-Small dataset is composed of 3 different files:

- `HI-Small_account.csv`, contains 518,581 accounts together with their features:
 - *Entity ID*: The ID of the account;
 - *Entity Name*: The name of the account;
 - *Bank Name*: The name of the bank holding the account;
 - *Bank ID*: The ID of the bank holding the account;
 - *Account Number*: The number of the account;

Figure ?? shows some examples of account records.

- `HI-Small_Trans.csv`, contains transactions features and binary labels:
 - *Timestamp*: The timestamp of the transaction;
 - *From Bank*: The ID of the sending bank;
 - *From Account*: The ID of the sending account;

	Bank Name	Bank ID	Account Number	Entity ID	Entity Name
0	Portugal Bank #4507	331579	80B779D80	80062E240	Sole Proprietorship #50438
1	Canada Bank #27	210	809D86900	800C998A0	Corporation #33520
2	UK Bank #33	21884	80812BE00	800C47F50	Partnership #35397
3	Germany Bank #4815	32742	81047F300	80096F0B0	Corporation #48813
4	National Bank of Harrisburg	127390	80BD8CF00	800FB8760	Corporation #889

Figure 4: IBM AMLSim account records

	Timestamp	From Bank	From Account	To Bank	To Account	Amount Received	Receiving Currency	Amount Paid	Payment Currency	Payment Format	Is Laundering	src	dst
0	2022-09-01 00:20:00	010	8000EBD30	010	8000EBD30	3697.34	US Dollar	3697.34	US Dollar	Reinvestment	0	010_8000EBD30	010_8000EBD30
1	2022-09-01 00:20:00	03208	8000F4580	001	8000F5340	0.01	US Dollar	0.01	US Dollar	Cheque	0	03208_8000F4580	001_8000F5340
2	2022-09-01 00:00:00	03209	8000F4670	03209	8000F4670	14675.57	US Dollar	14675.57	US Dollar	Reinvestment	0	03209_8000F4670	03209_8000F4670
3	2022-09-01 00:02:00	012	8000F5030	012	8000F5030	2806.97	US Dollar	2806.97	US Dollar	Reinvestment	0	012_8000F5030	012_8000F5030
4	2022-09-01 00:06:00	010	8000F5200	010	8000F5200	36682.97	US Dollar	36682.97	US Dollar	Reinvestment	0	010_8000F5200	010_8000F5200

Figure 5: IBM AMLSim transaction records

- *To Bank*: The ID of the receiving bank;
- *To Account*: The ID of the receiving account;
- *Amount Received*: The amount of money received by the recipient;
- *Receiving Currency*: The currency of the received amount (USD, EUR, etc.);
- *Amount Paid*: The amount of money paid by the sender;
- *Payment Currency*: The currency of the paid amount;
- *Payment Format*: The format/method of the payment;
- *Is Laundering*: Binary label indicating whether the transaction is illicit;
- *src*: Source node identifier;
- *dst*: Destination node identifier;

Figure ?? shows some examples of transaction records.

- `HI-Small_Patterns.txt`, lists the edges that are involved in ML activities together with the type of pattern they are involved in (multiclass labels). Examples of the 8 different types of ML patterns in IBM AMLSim are shown in Figure ??.

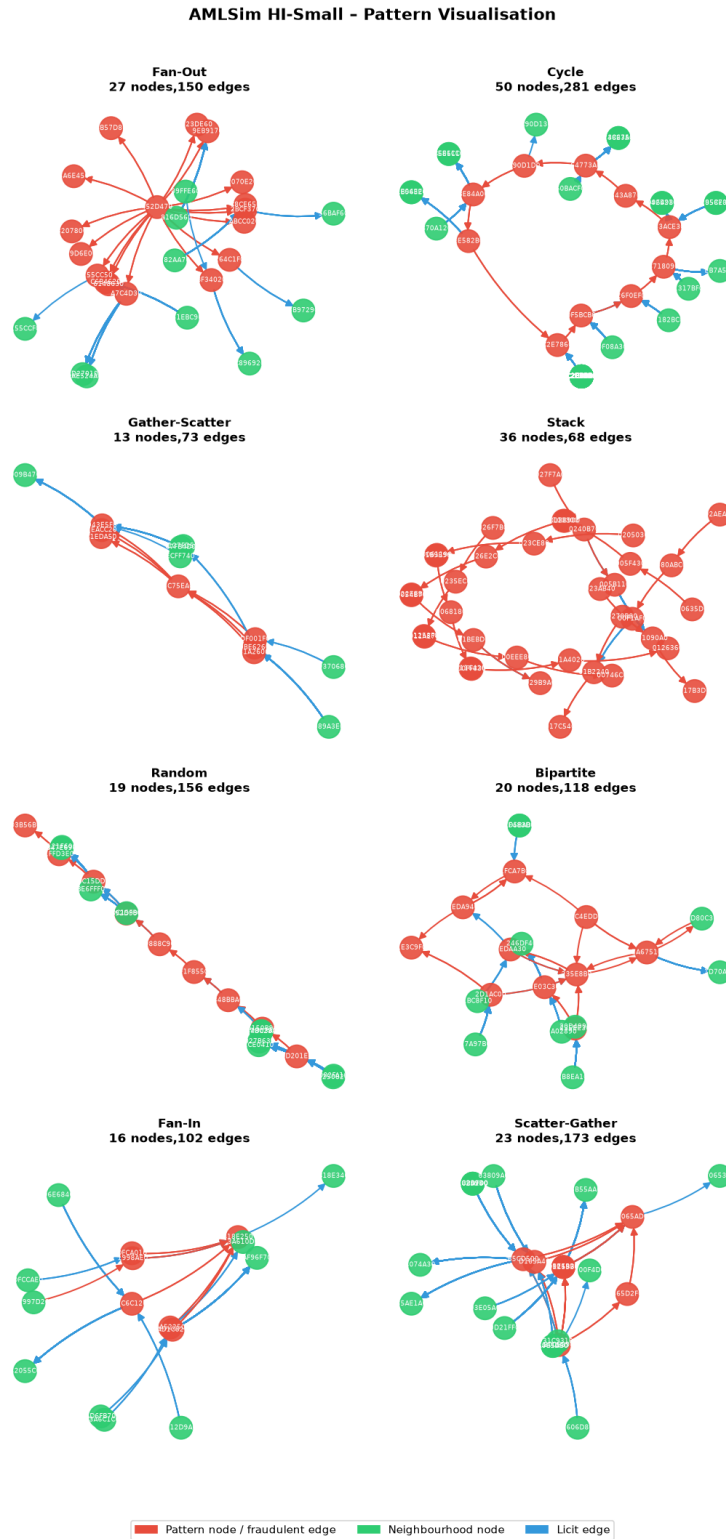


Figure 6: IBM AMLSim patterns examples

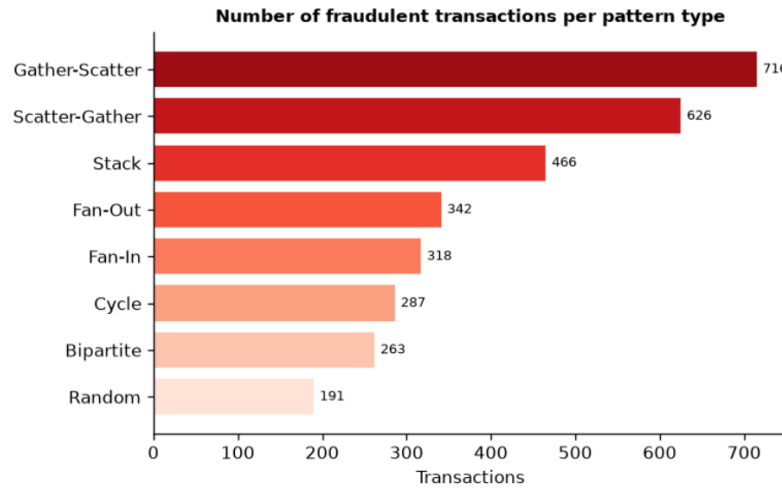


Figure 7: IBM AMLSim patterns distribution over fraudulent edges

2.3.2 Features analysis

In this section we provide a detailed analysis of the edges and nodes features distributions. Before diving into that we first take a look at the distribution of the labels in the dataset. There are 5,177 illicit transactions over a total of 5,078,345 Edges. Figure ??, provides the distribution of the multiclass label (type of pattern) over the fraudulent edges. It's important to notice that this information is provided only for 3,149 out of the 5,177 illicit transactions. This issue will be addressed in Section ??.

We now proceed by analyzing the edge features. First, we notice that *Amount Received* and *Amount Paid* are almost identical. From Figure ?? we can see that fraudulent transactions have the tendency to be bigger than licit ones.

Continuing our analysis on edge features, we notice that the most common currency is USD for both fraudulent and licit transactions; most of the currencies have a similar proportion distribution from both fraudulent and licit transactions. From Figure ?? we can see that there are two exceptions: Shekel, which is more prevalent in licit transactions, and Saudi Riyal, which is significantly more prevalent in fraudulent transactions. It's important to notice that the graph shows the proportion of each currency with respect to the total number of transactions in each class; Since there are way more licit than fraudulent transactions, having similar distributions means that, for every type of currency, licit transactions still represent the vast majority.

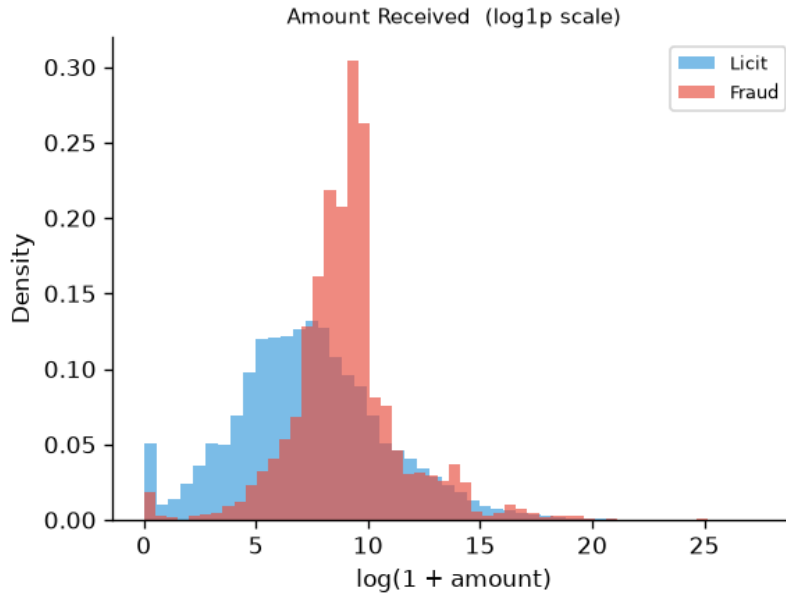


Figure 8: IBM AMLSim amounts distribution

Considering now the payment formats, by looking at Figure ?? we notice that ACH transactions are more frequently used for fraudulent transactions compared to other methods.

The last edge feature is *Timestamp*. Since working with timestamps can be tricky, we decided to extract to categorical features representing the hour of the day and the day of the week (Figures ??). Normally we would have also considered the week of the year and the year itself but, since IBM AMLSim Small only covers the span of a week, it would be irrelevant. That being said, it's important to mention that directly working with timestamps, albeit tricky, could lead to a more precise detection [Chapter ??].

Finally we take a look at the Node features. The only interesting one is *Bank Name*, Figure ?? shows the distribution of licit and illicit nodes over the 10 biggest banks.

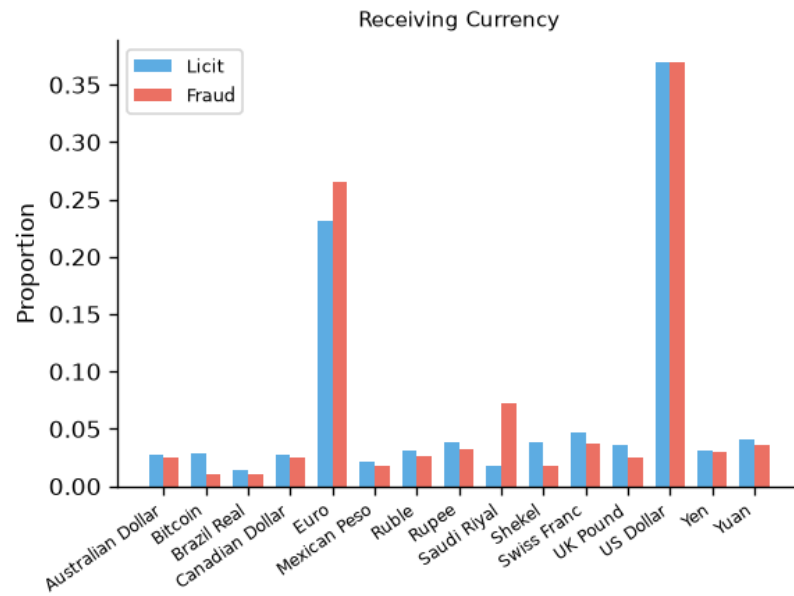


Figure 9: IBM AMLSim currencies distribution

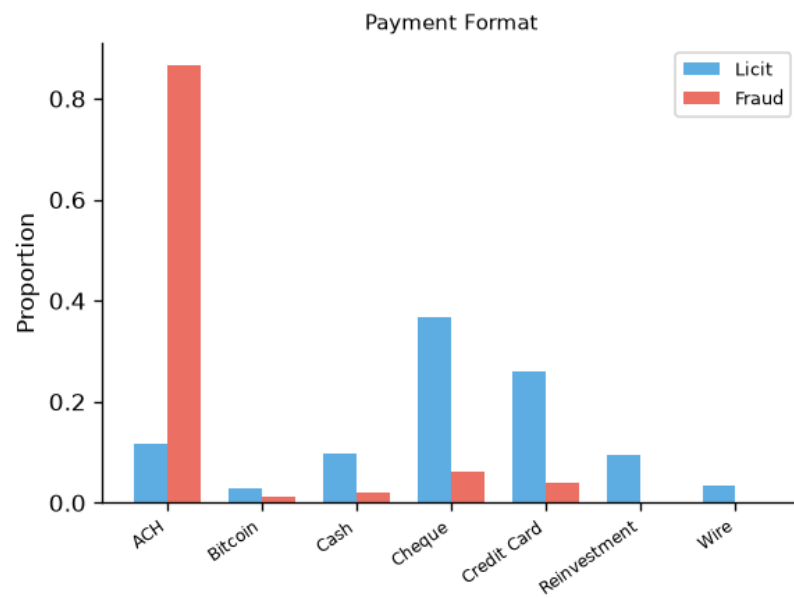


Figure 10: IBM AMLSim payment formats distribution

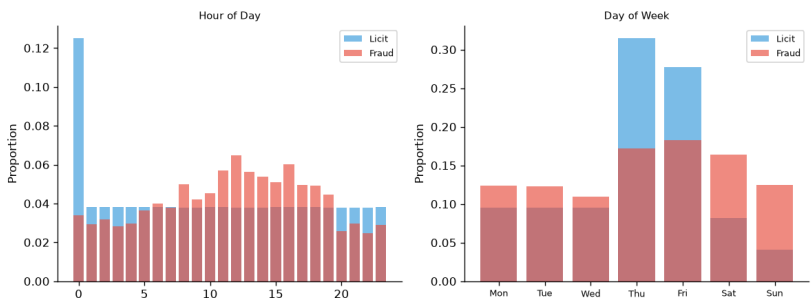


Figure 11: IBM AMLSim hour and day of the week distributions

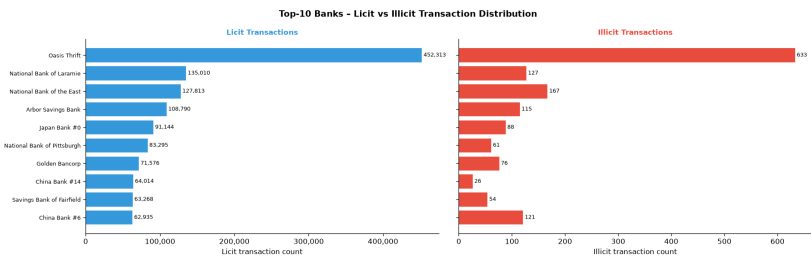


Figure 12: IBM AMLSim banks distribution

METHODOLOGY

This chapter describes the GNN architectures for edge-level money laundering detection proposed in this dissertation. We will propose a total of four different models; Graph Attention Networks (GAT, Section ??) and Graph Isomorphism Networks (GIN, Section ??) will be used as evaluation baselines; Finally, to mitigate the limitations (Section ??) of the two previous models, we will consider the architecture proposed in the paper Group-Aware Graph Neural Network [?] (Section ??) which, by aggregating nodes that are involved in suspicious transactions into gangs, and running a GNN on the obtained grouped graph, tries to detect money laundering organizations. We will consider two variants of the model one based on a GAT and one based on a GIN.

3.1 BUILDING THE FINANCIAL GRAPH

Before describing the models we define how the graph itself is built . Since the IBM AMLSim Dataset account (Node) features are very limited, we will leverage the transaction (Edge) features to derive the Node's ones.

Let $\mathcal{N}(i)$ be the set of neighbors of account i , e_{ij} the features vector of a transaction from account i to account j , y_{ij} the label of that transaction. The feature vector x_i for the account i and his ground truth label $\hat{y}_i$ are defined as follows.

$$x_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} e_{ij} \quad (3.1)$$

$$\hat{y}_i = \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} y_{ij} \quad (3.2)$$

It's important to mention that in case of qualitative edge features or multiclass edge labels, the mean will be computed on the one-hot encoded vectors.

3.2 GRAPH ATTENTION NETWORKS

As mentioned in Section ?? different types of GNNs are defined by different AGGREGATE and UPDATE functions.

Graph Attention Networks introduce learned attention coefficients to improve the fixed weight aggregation of classical graph convolutions, allowing the model to understand which neighbours are more relevant towards the classification task [?]. The AGGREGATE function is defined as:

$$\mathbf{m}_i^{(k)} = \sum_{t=1}^H \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k,t)} \mathbf{W}^{(k,t)} \mathbf{h}_j^{(k-1)} \quad (3.3)$$

where H is the number of attention heads, $\mathcal{N}(i)$ is the set of neighbors of node i , $\mathbf{W}^{(k,t)}$ is the weight matrix for the attention head t at layer k and $\alpha_{ij}^{(k,t)}$ is the attention coefficient between node i and its neighbor j at layer k and head t defined as

$$\alpha_{ij}^{(k,t)} = \frac{\exp(e_{ij}^{(k,t)})}{\sum_{l \in \mathcal{N}(i)} \exp(e_{il}^{(k,t)})} \quad (3.4)$$

where, in our case, the unnormalized attention scores $e_{ij}^{(k,t)}$ also rely on the edge features ℓ_{ij} of the edge (i, j) .

$$e_{ij}^{(k,t)} = \text{LeakyReLU} \left(\mathbf{a}^{(k,t)\top} \begin{bmatrix} \mathbf{W}^{(k,t)} \mathbf{h}_i^{(k-1)} & \mathbf{W}^{(k,t)} \mathbf{h}_j^{(k-1)} & \mathbf{W}_e^{(k,t)} \ell_{ij} \end{bmatrix} \right) \quad (3.5)$$

where $\mathbf{a}^{(k,t)}$, $\mathbf{W}^{(k,t)}$ and $\mathbf{W}_e^{(k,t)}$ are learnable parameters.

The UPDATE function is then defined as:

$$\mathbf{h}_i^{(k)} = \begin{cases} \text{ReLU}(\mathbf{m}_i^{(k)}) & k < L \\ \text{Id} \left(\frac{1}{H} \sum_{t=1}^H \sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(k,t)} \mathbf{W}^{(k,t)} \mathbf{h}_j^{(k-1)} \right) & k = L \end{cases} \quad (3.6)$$

Notice how the UPDATE function is different for the last layer. This is due to the fact that it averages the single heads outputs to produce an

embedding of dimension d_{out} instead of concatenating them; which is the case for intermediate layers.

The embeddings are then leveraged for edge classification. Let (i, j) be an edge, its final representation will be the concatenation of the embedding of the source node $h_i^{(L)}$, the embedding of the destination node $h_j^{(L)}$ and the feature vector ℓ_{ij} . $[h_i^{(L)}, h_j^{(L)}, \ell_{ij}]$ is then passed through a Multi-Layer Perceptron that computes the prediction.

3.3 GRAPH ISOMORPHISM NETWORKS

Graph Isomorphism Networks achieve the theoretical upper bound of expressive power for message passing GNNs [?]. By using a MLP in the update function instead of a simple linear transformation, they are able to match the expressive power of the 1-Weisfeiler-Lehman test (Definition ??). The AGGREGATE function is simply defined as:

$$m_i^{(k)} = \sum_{j \in \mathcal{N}(i)} h_j^{(k-1)} \quad (3.7)$$

The UPDATE function, which is what actually differentiate GIN from other GNNs, is defined as:

$$h_i^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \varepsilon^{(k)} \right) h_i^{(k-1)} + m_i^{(k)} \right) \quad (3.8)$$

where $\varepsilon^{(k)}$ is a learnable parameter. In our implementation, we used a two-layer MLP with a ReLU activation:

$$\text{MLP}(z) = W_2 \text{ReLU}(W_1 z + b_1) + b_2 \quad (3.9)$$

where W_1 , W_2 , b_1 and b_2 are learnable parameters. The embeddings are then leveraged for edge classification in the same way as for GAT.

3.4 LIMITATIONS OF LOCAL MESSAGE PASSING

GATs and GINs, both belonging to the family of local message passing GNNs, share common limitations.

3.4.1 Weisfeiler-Leman test upperbound

Theoretically it has been proved that the expressive power of local message passing GNNs is upper bounded by the 1-Weisfeiler-Leman test [?];

Definition 3.1 (Isomorphic Graphs). *Two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic if there exists a bijection $f : V \rightarrow V'$ such that for all $u, v \in V$, $(u, v) \in E \iff (f(u), f(v)) \in E'$.*

Definition 3.2 (1-Weisfeiler–Leman test). *The 1-Weisfeiler–Leman (1-WL) test is an algorithm to determine whether two graphs $G = (V, E)$ and $G' = (V', E')$ are isomorphic or not. It works by iteratively assigning colours to nodes according to the local structure of the graph. The process is defined as follows:*

1. **Initialization:** *Every node is initialised with the same color $c_i^{(0)} = c^{(0)} \forall i \in V, V'$;*
2. **Iteration:** *At each step k , each node updates its colour by collecting the colours of all its immediate neighbours and combining them with its current colour to create a new one:*

$$c_i^{(k)} = \text{HASH}\left(c_i^{(k-1)}, \left\{\left\{c_j^{(k-1)} : j \in \mathcal{N}(i)\right\}\right\}\right) \quad (3.10)$$

the HASH function guarantees that different combinations of colours will always produce a new distinct colour.

3. **Comparison:** *After each step, the algorithm compares the number of nodes for each colour in both graphs. If they are different, we are certain that the graphs are not isomorphic. Unfortunately, if the colours stop updating and the sets are still identical, the test is inconclusive; there is no guarantee that the two graphs are actually isomorphic. In Figure ?? we can see an example of non-isomorphic graphs that are not discriminated by the 1-WL test.*

This means that message passing GNNs are unable to discriminate certain non-isomorphic graph structures, limiting their ability to detect global topological patterns.

3.4.2 Oversmoothing

Growing the number of layers L in a GNN generally allows to capture higher order and more complex dependencies. However, as L grows, node embeddings tend to converge to a stationary point that is virtually identical for all nodes [?]. When this happens, the node embeddings lose their ability to represent different nodes and the model loses its discriminative power.

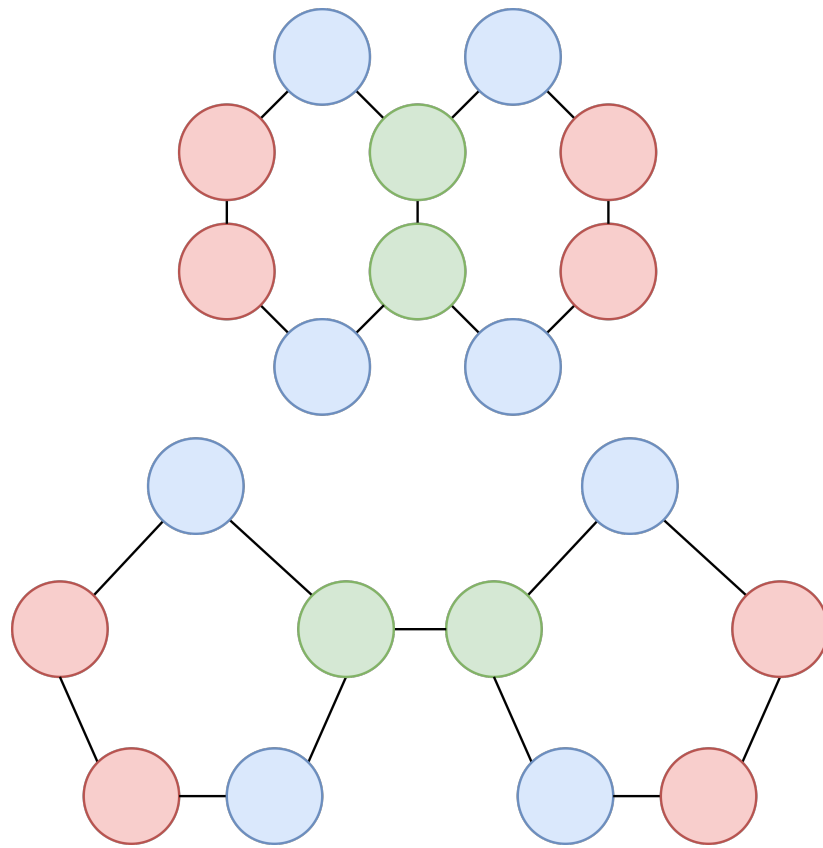


Figure 13: Non-isomorphic graphs that are not discriminated by the 1-WL test

3.4.3 Oversquashing

To capture interactions between nodes separated by a distance r , GNNs require at least r layers of message passing.

While the receptive field $|\mathcal{N}_r(v)|$ grows exponentially with r , the representation $h_v \in \mathbb{R}^d$ maintains its fixed capacity d . Consequently, propagating signals across multiple hops forces an exponentially increasing volume of neighborhood information into a fixed-size vector, creating an informational bottleneck.

This gets worse across graph bottlenecks, where signals from distant nodes u are heavily diluted and fail to influence the final embedding of v . [?].

To overcome those limits we will leverage Group-Aware GNNs (GAGNNs) [?] (Section ??), introducing a clustering mechanism to merge nodes that are suspected to be doing ML together and predicting their labels as a groups.

3.5 GROUP-AWARE GRAPH NEURAL NETWORKS

As introduced in Section ??, money laundering is inherently an organized crime. Illicit transactions are not isolated events scattered randomly across the graph, but are rather concentrated among clusters of colluding accounts acting as gangs [?]. Detecting fraud solely at the node or edge level tends to ignore this crucial group structure.

To overcome these limitations, we adopt the *Group-Aware Graph Neural Network* (GAGNN) architecture proposed by Cheng et al. [?]. GAGNN is a multi-component pipeline that can embed any of the previously described GNNs (GAT and GIN) as backbone and enriches them with two key mechanisms:

1. An *eMRF layer* (Edge Markov Random Field) that encourages nodes involved in the same fraud ring to receive similar embeddings.
2. A *Group Representation Layer* that explicitly detects and groups suspicious accounts into super-nodes, adding a community-level classification.

The overall GAGNN pipeline proceeds in five steps.

3.5.1 Community-Centric Encoder

The first step employs a standard GNN backbone (GAT or GIN) to transform the raw node feature matrix $X^{(1)} \in \mathbb{R}^{N \times d_{\text{feat}}}$ into enriched embeddings $X^{(2)} \in \mathbb{R}^{N \times d_{\text{out}}}$ by stacking L message passing layers;

$$X^{(2)} = f_{\text{GNN}}^{(L)} \left(\dots f_{\text{GNN}}^{(1)} \left(X^{(1)}, E \right) \dots \right) \quad (3.11)$$

where E denotes the set of edges. A classifier then uses these embeddings to produce a preliminary node-level prediction p_{fraud} , which will be utilized by the subsequent eMRF layer;

$$p_{\text{fraud}} = \sigma \left(W_c X^{(2)} \right) \in [0, 1]^N \quad (3.12)$$

where σ is the sigmoid activation function. Notice that p_{fraud} represents the probability of a node being illicit. Because the node classification step focuses strictly on detecting the presence of laundering, this prediction remains binary even in multiclass scenarios. The differentiation of specific laundering patterns is handled exclusively at the transaction level later in the pipeline.

3.5.2 Extended Markov Random Field Layer

The eMRF layer refines the embeddings $X^{(2)}$ by pulling together nodes that belong to the same community and repelling those that do not. This community similarity is derived from both the graph topology and the node attributes.

First, the topological similarity $\varepsilon(v_i, v_j)$ measures how much more connected nodes v_i and v_j are compared to the average connection density of a random graph:

$$\varepsilon(v_i, v_j) = \frac{d_i d_j}{2|E|} - a_{ij} \quad (3.13)$$

where d_i is the degree of node v_i , $|E|$ is the total number of edges, and a_{ij} is the adjacency indicator; $a_{ij} = 1$ if an edge exists between v_i and v_j and 0 otherwise. Next, the attribute similarity is computed as the normalized cosine similarity between the intermediate embeddings:

$$R_\zeta(v_i, v_j) = \text{Normalize} \left(\frac{h_i^{(2)\top} h_j^{(2)}}{\|h_i^{(2)}\| \|h_j^{(2)}\|} \right) \quad (3.14)$$

These two measures are linearly combined via a trade-off parameter β :

$$\gamma(v_i, v_j) = \beta \varepsilon(v_i, v_j) + (1 - \beta) R_z(v_i, v_j) \quad (3.15)$$

Nodes predicted to belong to the same class by the coarse classifier are attracted, while nodes of different classes are repelled. The pairwise potential $\Psi(v_i, v_j)$ is defined as:

$$\Psi(v_i, v_j) = (1 - 2\sigma(v_i, v_j)) \cdot \gamma(v_i, v_j) \quad (3.16)$$

where $\sigma(v_i, v_j) = \mathbf{1}[\lfloor p_{\text{fraud},i} + 0.5 \rfloor = \lfloor p_{\text{fraud},j} + 0.5 \rfloor]$ is a binary indicator of the same predicted class. Because the coarse node classifier is strictly binary, this indicator remains mathematically identical whether the overarching task is binary or multiclass.

Notice that our implementation differs mathematically from the original paper formulation, which used an exponential sign term $(-1)^{\sigma(v_i, v_j)}$. Since the derivative of a^x relies on $\ln(a)$, computing $\ln(-1)$ is mathematically undefined and causes PyTorch to output NaN gradients during the backward pass. We solved this by using the equivalent linear mapping $(1 - 2\sigma)$. Furthermore, since the hard threshold $\lfloor \cdot + 0.5 \rfloor$ is not differentiable, we utilize a Straight-Through Estimator (STE). During the forward pass, the model uses the hard indicator, while during the backward pass, gradients are propagated through a soft continuous approximation.

Finally, the refined embeddings $X^{(3)}$ are computed using a mean-field update:

$$X^{(3)} = X^{(2)} - \Psi X^{(2)} H^{(2)} \quad (3.17)$$

where $H^{(2)}$ is a learnable weight matrix.

3.5.3 Node-Level Classification

The refined embeddings $X^{(3)}$ are mapped to node-level classification logits by a two-layer Multi-Layer Perceptron (MLP):

$$p_{\text{node}} = \text{MLP}_{\text{node}}(X^{(3)}) \quad (3.18)$$

These logits provide the first of three supervision signals used to train the network.

3.5.4 Group Representation Layer

This layer classifies individual edges and discovers the latent groups of colluding accounts. First, for each directed edge (i, j) , the layer concatenates the source and destination embeddings with the raw edge features to compute the transaction probability:

$$p_{\text{trans}}(e_{ij}) = \text{MLP}_{\text{edge}}\left([X_i^{(3)} \parallel X_j^{(3)} \parallel \ell_{ij}]\right) \quad (3.19)$$

Then we used this distribution to construct the communities graph. Considering an edge e_{ij} , the two accounts i and j will be grouped together with a probability equal to their predicted illicit probability $p_{\text{trans}}(e_{ij})$ or, if the edge is in the training set, we will simply use its label to decide whether to group together the two nodes or not. In the case of a multi-class classification task, this Bernoulli probability is computed as $1 - p_{\text{licit}}(e_{ij})$, effectively considering all illicit classes together. The resulting groups are then collapsed into single super-nodes whose feature vector is the mean of its members' original features. Two communities will be connected by an edge if at least one node from one community have an edge with a node in the other community; the edge features will be dropped mantaing only the information about the connectivity.

$$\hat{x}_g = \frac{1}{|\mathcal{G}_i|} \sum_{v \in \mathcal{G}_i} x_v \quad (3.20)$$

Each group $\mathcal{G}$ is considered illicit if it contains at least one illicit account, licit otherwise. Formally we define the group label as:

$$\hat{y}_{\mathcal{G}} = \begin{cases} 1 & \text{if } \exists V \mid V \text{ is ML} \wedge V \in \mathcal{G} \\ 0 & \text{otherwise} \end{cases} \quad (3.21)$$

3.5.5 Group-Level Encoding and Prediction

The same Community-Centric Encoder from Step 1 is reused on this new group graph $\hat{\mathcal{G}}$ with shared weights:

$$\hat{X}^{(3)} = \text{CommunityCentricEncoder}(\hat{X}, \hat{\mathcal{E}}) \quad (3.22)$$

$$p_{\text{group}} = \sigma\left(W_c \hat{X}^{(3)}\right) \quad (3.23)$$

where $\hat{X}^{(3)}$ is the resulting group-level embedding, and p_{group} is the final illicit probability prediction obtained by performing a classification step on these embeddings. Sharing weights enforces consistency between the node and group levels while keeping the parameter count low.

3.5.6 Multi-Task Loss

Unlike standard GNNs that optimize a single objective, GAGNN relies on three supervised signals. The total loss $\mathcal{L}$ is a weighted sum of the node-level, transaction-level, and group-level losses:

$$\mathcal{L} = c_1 \mathcal{L}_{\text{group}} + c_2 \mathcal{L}_{\text{node}} + c_3 \mathcal{L}_{\text{trans}} \quad (3.24)$$

where $c_1, c_2, c_3 \in \mathbb{R}^+$ are hyperparameters and $\mathcal{L}_{\text{group}}$, $\mathcal{L}_{\text{node}}$ and $\mathcal{L}_{\text{trans}}$ are the group-level, node-level, and transaction-level losses, respectively. Since nodes and groups are strictly classified as illicit or legitimate, their losses are always weighted Binary Cross-Entropy (BCE) functions:

$$\mathcal{L}_{\text{node}} = \text{BCE}_w(p_{\text{node}}, y_{\text{node}}) \quad (3.25)$$

$$\mathcal{L}_{\text{group}} = \text{BCE}_w(p_{\text{group}}, y_{\text{group}}) \quad (3.26)$$

For the transaction classification, $\mathcal{L}_{\text{trans}}$ is a BCE loss for binary tasks, and a Cross-Entropy (CE) loss for multiclass classification.

$$\mathcal{L}_{\text{trans}} = \begin{cases} \text{BCE}_w(p_{\text{trans}}, y_{\text{trans}}) & \text{if binary task} \\ \text{CE}_w(p_{\text{trans}}, y_{\text{trans}}) & \text{if multiclass task} \end{cases} \quad (3.27)$$

The positive-class weight w is tuned to penalize false negatives, in the finance sector detecting most of the frauds is more important than maximizing the accuracy. By optimizing this joint loss, the model learns to identify money laundering at the individual, transaction, and community levels simultaneously.

3.6 EXPERIMENTAL SETTINGS

3.6.1 Data Preparation

In regard to enhance the performance of the models we perform some preprocessing steps on the dataset. First of all, categorical features need to be converted in a numerical format; we opted for One-Hot Encoding,

which works particularly well for our architecture since, when we average the features to create the nodes and groups representations, the one-hot vectors translate into proportions of the different categories providing like that meaningful information.

Secondly, we elaborated the timestamp by extrapolating from it the hour of the day, using cyclic encoding, and the day of the week, using a one-hot encoder. The cyclic representation is fundamental to preserve the daily periodicity of transactions.

We then normalized the financial amounts. Since money laundering datasets present a high number of outliers standard normalization techniques often tend to move the value of the legitimate transactions close to zero. For this reason a Robust Scaler was preferred to the standard one. It normalizes features using the median and the interquartile range (IQR), according to the following formula:

$$x_{\text{scaled}} = \frac{x - Q_2(x)}{Q_3(x) - Q_1(x)} \quad (3.28)$$

where $Q_2(x)$ is the median, and $Q_1(x)$ and $Q_3(x)$ are the 25th and 75th percentiles of the feature values. This prevents extreme values from distorting the distribution of the data.

Finally, we extrapolated the multiclass labels from `HI-Small_Patterns.txt` obtaining the categorical feature. As before, we then apply one-hot encoding to allow the model to understand it.

3.6.2 Train, Patience, Validation and Test Split

Anti Money Laundering algorithms are usually performed on the new incoming transactions to detect fraudulent activity. This means that the models must learn from the past to predict the future. For this reason, splitting the dataset randomly would inevitably cause future information to leak into the past. To avoid that we decided to split our dataset by using 3 temporal thresholds t_1, t_2, t_3 that define the training, patience, validation and test sets as follows.

$$\mathcal{D}_{\text{train}} = \{d \in \mathcal{D} | t(d) \leq t_1\} \quad (3.29)$$

$$\mathcal{D}_{\text{patience}} = \{d \in \mathcal{D} | t_1 < t(d) \leq t_2\} \quad (3.30)$$

$$\mathcal{D}_{\text{val}} = \{d \in \mathcal{D} | t_2 < t(d) \leq t_3\} \quad (3.31)$$

$$\mathcal{D}_{\text{test}} = \{d \in \mathcal{D} | t(d) > t_3\} \quad (3.32)$$

where $t(d)$ is the timestamp of the transaction d , and t_1, t_2 and t_3 are specially picked so that $|\mathcal{D}_{\text{train}}| \approx 0.7|\mathcal{D}|$, $|\mathcal{D}_{\text{patience}}| \approx 0.075|\mathcal{D}|$, $|\mathcal{D}_{\text{val}}| \approx$

$0.075|\mathcal{D}|$ and $|\mathcal{D}_{\text{test}}| \approx 0.15|\mathcal{D}|$. The patience set is exclusively used to trigger early stopping during both model selection and training. The validation set is instead kept strictly for evaluating and comparing models during the selection phase, preventing the evaluation metric from being biased by the early stopping mechanism.

During the training phase, since we no longer need 4 different subsets, the train set becomes $\mathcal{D}_{\text{train}} \leftarrow \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{patience}}$, the validation will be used for the early stopping $\mathcal{D}_{\text{patience}} \leftarrow \mathcal{D}_{\text{val}}$ and the test set remains untouched to provide an unbiased evaluation of the model performance. This ensures that the model always trains on a contiguous block of time and prevents data from the future to leak into the past.

It's important to mention that, in order to prevent data leakage, the Robust Scaler was fitted exclusively on the transactions belonging to the training split, and the resulting parameters (Q_1, Q_2, Q_3) were then used to independently transform the patience, validation and test sets.

Finally, in order to address the class imbalance, we decided to apply downsampling on the training set. For the binary task, the majority class (legitimate accounts) is downsampled to match the number of illicit accounts, achieving a 1:1 ratio. For the multiclass task, the legitimate class is downsampled to match the total sum of all illicit instances across the different illicit patterns. This achieves a 1:1 ratio between legitimate and overall illicit samples, while preserving the natural distribution and relative frequencies among the specific illicit classes. The validations and test sets remain untouched.

Test set problem

By analysing the test set we noticed a flaw in the IBM AMLSim dataset. The amount of ML transactions suddently increases in the last rows, corresponding to the newer transactions, of the dataset. Because of that we initially ended up with a test set that didn't really represented the real world scenario. The distribution of the ML attempts in the last rows is displayed in Table ?? . For this reason we decided to get rid of the last 5000 rows by re-defining the test set as:

$$\mathcal{D}_{\text{test}} = \{d \in \mathcal{D} | t_3 < t(d) \leq t_4\} \quad (3.33)$$

where t_4 is specially chosen to achive the objective.

Interval	Percentage of ML transactions
Last 100,000 rows	0.9%
Last 10,000 rows	6.7%
Last 5,000 rows	13.2%
Last 1,000 rows	59.0%

Table 1: ML transactions distribution in the last rows of the dataset

3.7 MODEL SELECTION AND TRAINING SETTINGS

In this section we describe and explain our choices for the training settings and the models' hyperparameters. We first discuss the general settings shared amongst all the experiments, then we deep dive into the architectural choices and hyperparameter tuning for each class of models we evaluated.

3.7.1 *Optimizer*

As suggested by Cheng et al. [?], we decided to adopt the Adam optimizer [?] as the optimization algorithm for all our models. The original implementation suggested a learning rate of 0.001 and a batch size of 128. With our dataset these settings led to a very unstable convergence, therefore we decided to lower the learning rate to $\eta = 0.0001$ which, together with gradient clipping at a maximum norm of 5.0, ensured smoother and more reliable training dynamics. Furthermore we also changed the batch size to 256 to better exploit the GPU usage.

3.7.2 *Subgraph Sampling*

Due to the prohibitive dimension of the financial graph, fitting the models directly to the entirety of it is not a feasible option in both terms of memory and time.

To address this, we employed a mini-batch subgraph sampling strategy using `NeighborLoader` from PyTorch Geometric [?]. The loader operates by sampling a fixed number of neighbors per hop. This is governed by the `num_neighbors = [a, b, c, ...]` parameter. Meaning that for each target node in the batch, the loader samples `a` of its immediate neighbors; for each of those `a` sampled neighbors, it further samples `b` of their neighbors

and so on. In our experiments, we set $\text{num_neighbors} = [15, 10, 10]$ across $L = 2$ or $L = 3$ hops depending on the model architecture. This preserves the local graph topology while keeping the computational cost manageable.

This sampling strategy is only used during training. For validation and testing, we set $\text{num_neighbors} = [-1, -1, -1]$, meaning that all neighbors are included. This guarantees an unbiased evaluation of the model’s performance on the realistic, imbalanced data.

3.7.3 Early Stopping

To prevent overfitting we employed an early stopping mechanism that monitors the loss on the patience set $\mathcal{D}_{\text{patience}}$. Training is stopped when no improvement is observed for a specified number of consecutive epochs (the *patience*). During the model selection phases, we set a maximum of 3000 epochs with a patience value of $p_{\text{selection}} = 100$; during the final training phase, we set a maximum of 6000 epochs with a patience value of $p_{\text{training}} = 150$, to allow the model to fully converge on the selected hyperparameters. These settings are shared across all the experiments.

3.7.4 Hyperparameters

We begin by listing the hyperparameters that will need to be tuned across all the different models.

LOSS FUNCTION HYPERPARAMETERS

- c_1 : Weight for the group-level loss $\mathcal{L}_{\text{group}}$.
- c_2 : Weight for the node-level loss $\mathcal{L}_{\text{node}}$.
- c_3 : Weight for the transaction-level loss $\mathcal{L}_{\text{trans}}$.
- w (*laundry_weight*): Positive-class weight used in the BCE loss.

GAGNN HYPERPARAMETERS

- β : Trade-off parameter in the Extended Markov Random Field (eMRF) layer that balances the influence of topological similarity ϵ and attribute similarity R_ζ (Section ??).

- `mlp_hidden_dim`: Dimension of the hidden layers in the MLP_{node} , used to map the refined embeddings $X^{(3)}$ to node-level classification logits (Section ??).
- `nn_t_hidden_dim`: Dimension of the hidden layers in the MLP_{edge} , used to compute the transaction probability p_{trans} (Section ??).
- `out_dim`: Dimension d_{out} of the enriched embeddings $X^{(2)}$ (Section ??).

GAT HYPERPARAMETERS

- `hidden_dim`: Dimension of the hidden layers in the message passing steps.
- `L (num_layers)`: Number of message passing layers, determining the multi-hop receptive field of the GNN.
- `H (heads)`: Number of attention heads.

GIN HYPERPARAMETERS

- `hidden_dim`: Dimension of the hidden layers in the message passing steps.
- `L (num_layers)`: Number of message passing layers, determining the multi-hop receptive field of the GNN.
- `gin_mlp_hidden`: Dimension of the hidden layers of the MLP used inside the GIN UPDATE function (Section ??).

Due to the very high number of hyperparameters that need to be tuned, we decided to split the model selection process into two distinct phases. The first one concentrates on finding the best values for the parameters involved in the loss function by keeping the architectural hyperparameters fixed to reference values suggested by Cheng et al. [?]. The second phase instead focuses on tuning the architectural hyperparameters (hidden dimension, number of layers, number of attention heads, etc.) keeping the loss function weights fixed to the values found in the first one.

Loss Function Hyperparameters

In this phase we search for the optimal values of the parameters c_1, c_2, c_3 and w (used only in the binary task), involved in the loss functions.

Parameter	Value	Rationale
L	2	original paper value
H	5	original paper value
β	0.44	original paper value
hidden_dim	64	original paper value
nn_t_hidden_dim	128	original paper value
out_dim	128	assumed as hidden_dim *2
mlp_hidden_dim	128	assumed as hidden_dim *2

Table 2: Fixed architectural hyperparameters for Phase 1

Parameter	Searched values	Rationale
c_1	{0.75}	Fixed
c_2	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
c_3	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
w (laundry weight)	{1.0, 2.0}	to penalize false negatives

Table 3: Grid search for binary loss function hyperparameters

To save computational resources, the grid search for the loss weights is performed exclusively on the GAGNN model with the GAT backbone, assuming that the optimal weights found for GAT apply directly to the GIN backbone as well for their respective tasks. During this phase, all architectural hyperparameters were kept fixed to reference values suggested by Cheng et al. [?]; when not explicitly stated we opted for feasible and reasonable values according to the literature. The fixed architectural hyperparameters are reported in Table ??.

A critical design choice in this phase concerns the evaluation metric. Since the validation loss $\mathcal{L}_{\text{val}}$ is directly affected by the parameters c_1, c_2, c_3 and w , using it as a selection criterion would be a mistake. We therefore leveraged the macro F1-score on the validation set as the model selection criterion. Validation loss still served to trigger the early stopping.

The hyperparameters grids for binary and multiclass tasks are reported in Table ?? and Table ?? respectively.

For the multiclass task the weight w is replaced by automatically computed per-class weights using sklearn with `class_weight='balanced'`.

Parameter	Searched values	Rationale
c_1	{0.75}	Fixed
c_2	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75
c_3	{0.5, 0.6, 0.75, 0.9, 1.0}	grid around 0.75

Table 4: Grid search for multiclass loss function hyperparameters

We denote the optimal values produced by the grid searches as $c_1^{*binary}$, $c_2^{*binary}$, $c_3^{*binary}$, w^* and c_1^{*multi} , c_2^{*multi} , c_3^{*multi} .

Architecture Hyperparameters

The second phase consists in finding the optimal values for the architectural hyperparameters, keeping the loss function weights fixed to the values found in the first phase. This allows to use the *validation loss* $\mathcal{L}_{val}$ instead of the validation F1-score as selection criterion; this is a better choice in this case since the F1-score metric is defined by strict decisions boundaries and therefore is not continuous and more sensitive to small fluctuations.

We will perform four different model selections, one for each experiment: GAGNN with GAT backbone for the binary task, GAGNN with GIN backbone for the binary task, GAGNN with GAT backbone for the multiclass task and GAGNN with GIN backbone for the multiclass task. Two hyperparameters grid are used, since they are shared between binary and multiclass tasks with the same backbone GNN. As possible values we chose options around those of the reference paper; the hyperparameters grids for GAT and GIN backbones are reported in Table ?? and Table ?? respectively.

Trying more parameters, especially for the number of layers, number of heads, β and coefficients of the loss function could lead to much better results. However, due to the limitations on the computational resources, we had to limit the grid search to a reasonable number of combinations. We will discuss this further in Section ??.

Baseline Models Hyperparameters

For the baseline models (GIN and GAT) we used as hyperparameters the same values obtained for the internal GNN encoder of their respective GAGNN counterpart. This choice stems from the need to understand how much the GAGNN architecture enhances the GNNs capacity of

Parameter	Searched values
H	5
L	[2,3]
β	0.44
hidden_dim	[64, 128]
nn_t_hidden_dim	[128, 256]
out_dim	[128, 256]
mlp_hidden_dim	[128, 256]

Table 5: Grid search for GAGNN with GAT

Parameter	Searched values
L	[2,3]
β	0.44
hidden_dim	[64, 128]
nn_t_hidden_dim	[128, 256]
out_dim	[128, 256]
mlp_hidden_dim	[128, 256]
gin_mlp_hidden	[128, 256]

Table 6: Grid search for GAGNN with GIN

detecting fraudulent transactions, without the confounding factor of different hyperparameter settings.

RESULTS

In this chapter we present and discuss the results obtained by the previously described models on the dataset IBM AMLSim small [?]. We will first present the results of model selection (Sections ??, ??), to then proceed by analysing the results of the GAGNN models (Section ??) and comparing them to the baseline models to evaluate the effectiveness of our proposed method.

4.1 HYPERPARAMETER SELECTION RESULTS

As described in Section ?? and Section ??, the model selection process is splitted into two phases to mitigate the computational cost of the grid search. As we will see, this will come at the cost of a sub-optimal model but due to the limitations of our hardware this is still the best option.

4.1.1 Loss Function Hyperparameters

We began by performing the grid search on the loss function parameters for both binary and multiclass tasks using the GAGNN with GAT backbone. Table ?? and Table ?? show the optimal hyperparameters found by this process by optimizing the macro F1-score on the validation set.

As we can see in both binary and multiclass tasks the weight associated with the edge loss c_3 is the highest value. Making the edge loss the more significant to the model performance. Especially in the multiclass task, this is expected since the edge loss is the one that predicts the actual target variable, meaning the patterns, whilst the node loss and the group loss are only based on binary classification.

Another interesting observation is that, in the binary task, the weight associated with the node loss c_2 is lower than the one for the group loss c_1 . This suggests that the idea of trying to force the model to predict money

Parameter	Optimal Value
c_1	0.75
c_2	0.5
c_3	1.0
w (laundry weight)	2

Table 7: Optimal loss function hyperparameters for the binary task

laundering at a community level might be a good one. The aggregation into communities allows the model to indirectly expand his receptive field without adding more layers to the GNN architecture, this should mitigate the cances of incurring into oversmoothing.

In the multiclass this is no longer true, my hipotesis is that while the community prediction helps us detecting whether a community is involved in money laundering or not, it does not help us to detect the exact patterns; once the fraudulent nodes are merged into communities we can no longer understand what type of pattern generated that community and therefore the model loses the ability to predict it.

Finally, note how in the binary task the best parameters for c_2 , c_3 and w lay at the border of the hyperparameter search space suggesting that future works should further investigate these values.

Parameter	Optimal Value
c_1	0.75
c_2	0.75
c_3	0.9

Table 8: Optimal loss function hyperparameters for the multiclass task

4.1.2 Architectural Hyperparameters

In the second phase of model selection, we runned 4 separate grid searches for the GAGNN-GAT (Binary), GAGNN-GAT (Multiclass), GAGNN-GIN (Binary) and GAGNN-GIN (Multiclass) architectures to find the optimal hyperparameters for each of them. The results are displayed in Table ??, ??, ?? and ?? respectively.

The first thing to notice is that, in case of GAGNN-GAT (Binary), which is the same task tackled by the paper [?], although on a different dataset,

Parameter	Optimal Value
H	5
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128

Table 9: Optimal architectural hyperparameters for GAGNN with GAT (Binary task)

we found different optimal parameters. In particular they mention as best parameters `hidden_dim = 64` and `nn_t_hidden_dim = 128`, which differ from our best parameters `hidden_dim = 128` and `nn_t_hidden_dim = 256`. The number of layers `L` is the same as the one found in [?].

For the number of attention heads and the value of β we trusted the paper and used the same hyperparameters due to the high computational cost that would have come with their tuning. That being said, the differences in `hidden_dim` and `nn_t_hidden_dim` suggests that tuning all the architectural hyperparameters could lead to big improvements in the model performance.

Another interesting observation is that for both GAGNN-GIN and GAGNN-GAT the optimal parameters for the multiclass task suggest the need of a bigger model compared to the binary task. This is expected since usually more complex tasks require more complex models. As we will see Section ??, those bigger models led to overfitting suggesting, in case of multiclass classification, the need of a bigger dataset to successfully complete the task. Due to our hardware limitations we couldn't afford to do that, but future works should consider using the IBM AMLSim Large Dataset to overcome this issue.

Finally we notice how in GAGNN-GIN (Multiclass) the optimal number of layers is 3 which is different from the 2 of the binary case. This results together with the others displayed in table ?? are to be considered with caution; as we will see GAGNN-GIN for the multiclass task didn't converge during the training and therefore the results are not reliable.

Parameter	Optimal Value
H	5
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	256
mlp_hidden_dim	256

Table 10: Optimal architectural hyperparameters for GAGNN with GAT (Multi-class task)

Parameter	Optimal Value
L	2
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128
gin_mlp_hidden	256

Table 11: Optimal architectural hyperparameters for GAGNN with GIN (Binary task)

Parameter	Optimal Value
L	3
β	0.44
hidden_dim	128
nn_t_hidden_dim	256
out_dim	128
mlp_hidden_dim	128
gin_mlp_hidden	256

Table 12: Optimal architectural hyperparameters for GAGNN with GIN (Multi-class task)

4.2 MODELS RESULTS

In this section we present and discuss the results obtained by training GAGNN with GAT and GIN backbones on both binary and multiclass classification tasks and we compare them with the results obtained by training standard GNNs on the same tasks.

As expected the binary task was easier to solve. Due to our hardware limitations we cannot afford to use a larger dataset that would allow the training of bigger models and with that better performances on the multiclass task. The question that we will try to answer is whether the GAGNN architecture could be useful also for multiclass detection or if it only performs well on the binary task.

4.2.1 General considerations

We begin by providing some considerations regarding training dynamics that are common to all the models.

As we can see in Figures ??, ??, ??, ??, ??, ?? and ??, the training loss and the patience loss, albeit less for the latter, tend to be a bit noisy. This is mainly due to the fact that the losses are computed on mini batches of 256 nodes instead of on the entire dataset. Furthermore, while it is true that for the validation loss we consider all the neighborhood of the sampled nodes (`num_neighbors = [-1,-1,-1]`); for the training mini-batches, in regard to reduce the computational cost, we only considered 15 of the adjacent neighbors, 10 neighbors for each of those 15 and, in case of a 3 layer GNN, another 10 neighbors for each of those 10×15 (`num_neighbors = [15, 10, 10]`). This introduces further noise in the loss computation and it's probably the reason why in some experiments the training loss is not as smooth as we expect it to be.

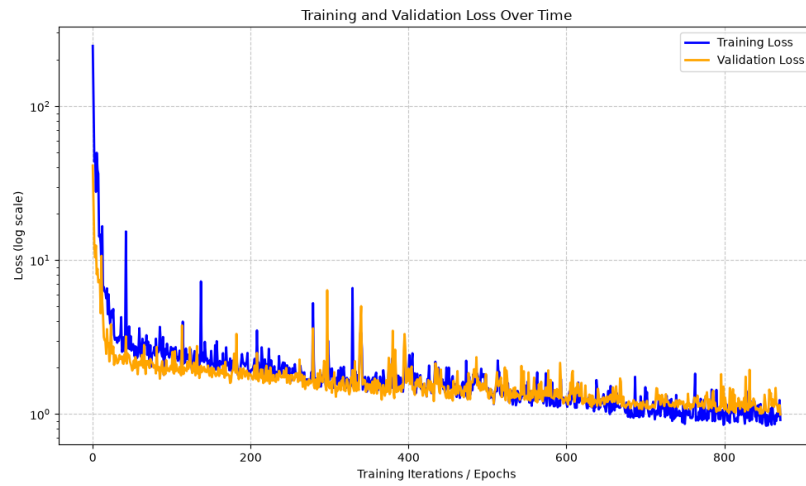


Figure 14: Training and patience loss for GAGNN-GAT (Binary task)

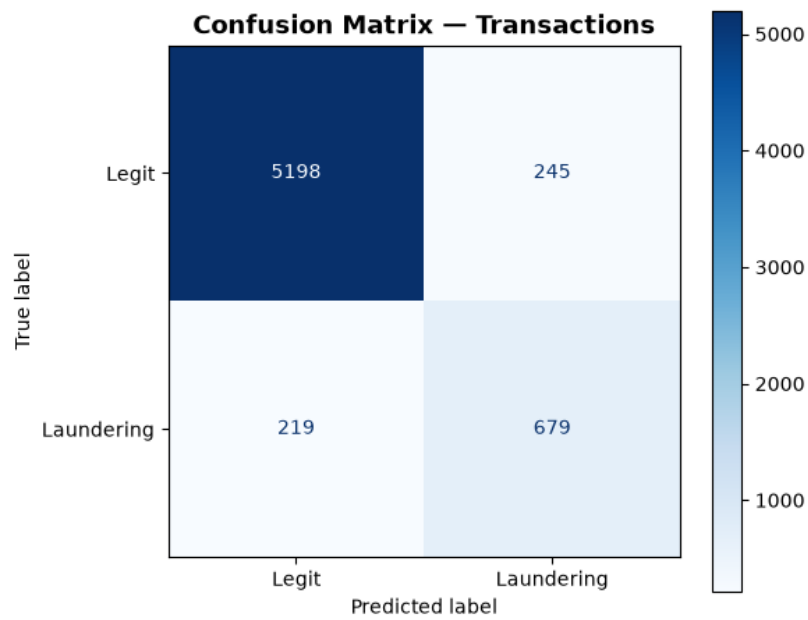


Figure 15: Confusion matrix for GAGNN-GAT (Binary task)

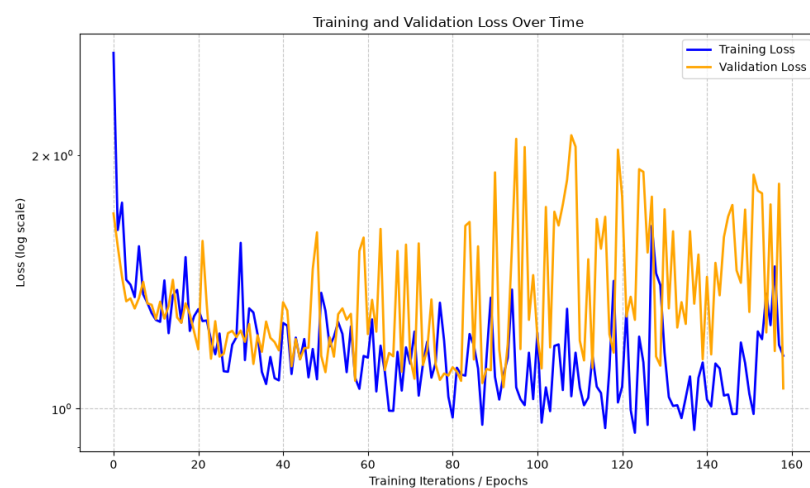


Figure 16: Training and patience loss for GAGNN-GIN (Binary task)

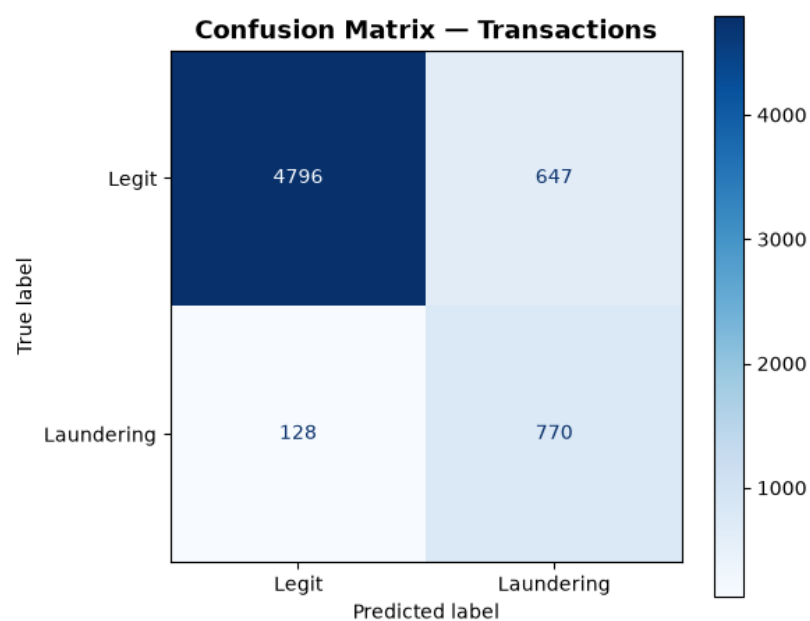


Figure 17: Confusion matrix for GAGNN-GIN (Binary task)

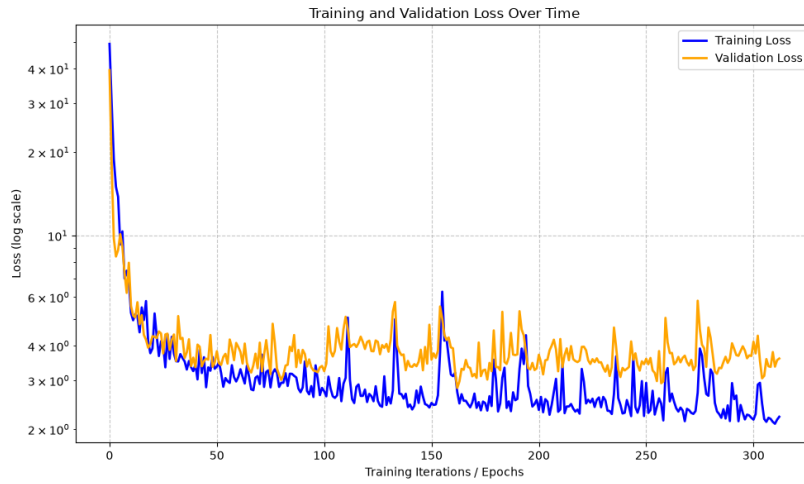


Figure 18: Training and patience loss for GAGNN-GAT (Multiclass task)

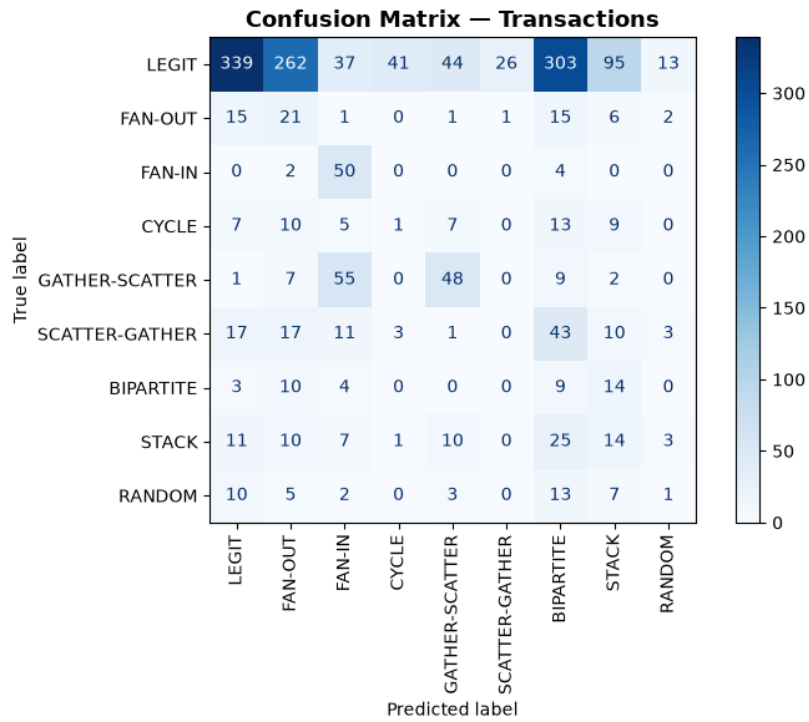


Figure 19: Confusion matrix for GAGNN-GAT (Multiclass task)

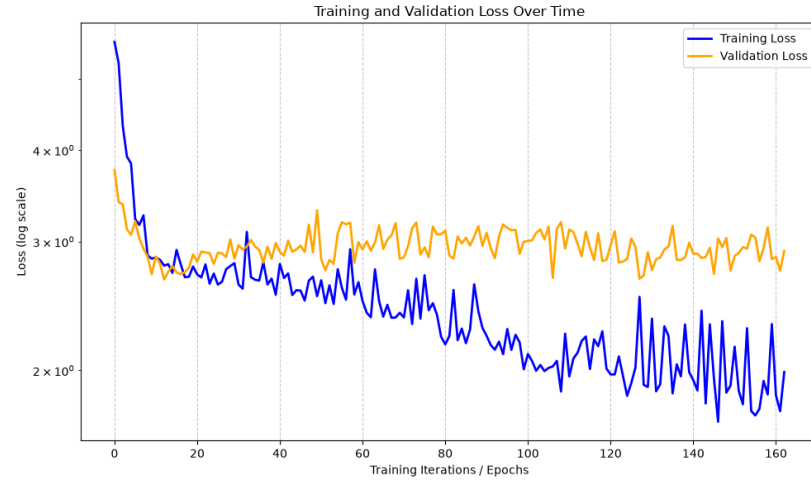


Figure 20: Training and patience loss for GAGNN-GIN (Multiclass task)

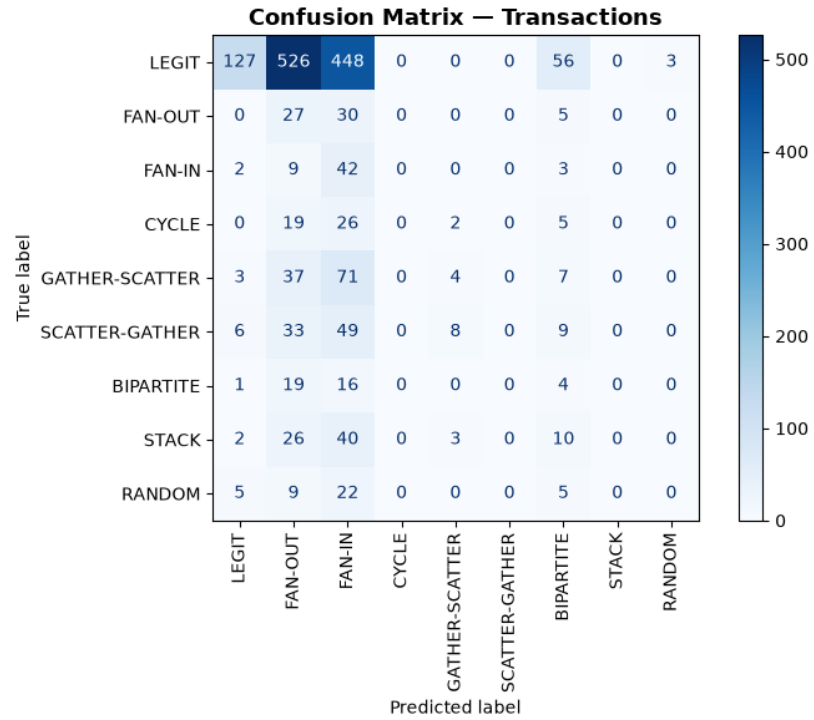


Figure 21: Confusion matrix for GAGNN-GIN (Multiclass task)

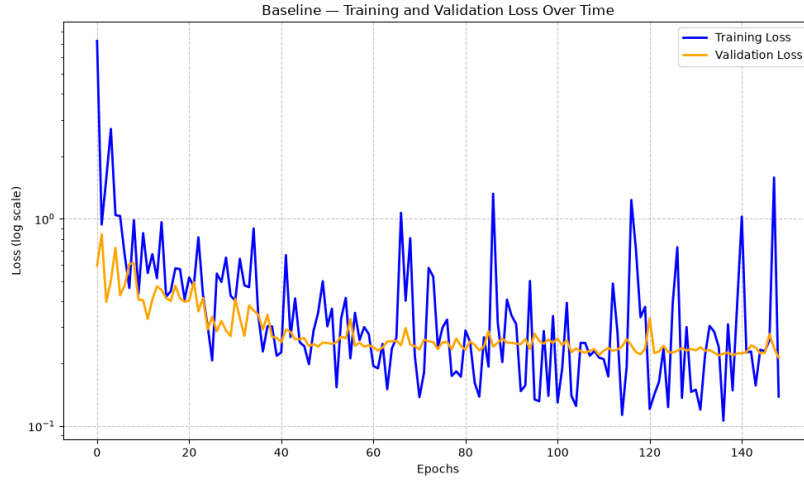


Figure 22: Training and patience loss for GAT baseline (Binary task)

4.2.2 *GAGNN with GAT (Binary Classification)*

4.2.3 *GAGNN with GAT (Multiclass Classification)*

4.2.4 *GAGNN with GIN (Binary Classification)*

4.2.5 *GAGNN with GIN (Multiclass Classification)*

4.2.6 *Graph Attention Network (Binary Classification)*

4.2.7 *Graph Isomorphism Network (Binary Classification)*

4.2.8 *Graph Attention Network (Multiclass Classification)*

4.2.9 *Graph Isomorphism Network (Multiclass Classification)*

4.3 LIMITATIONS

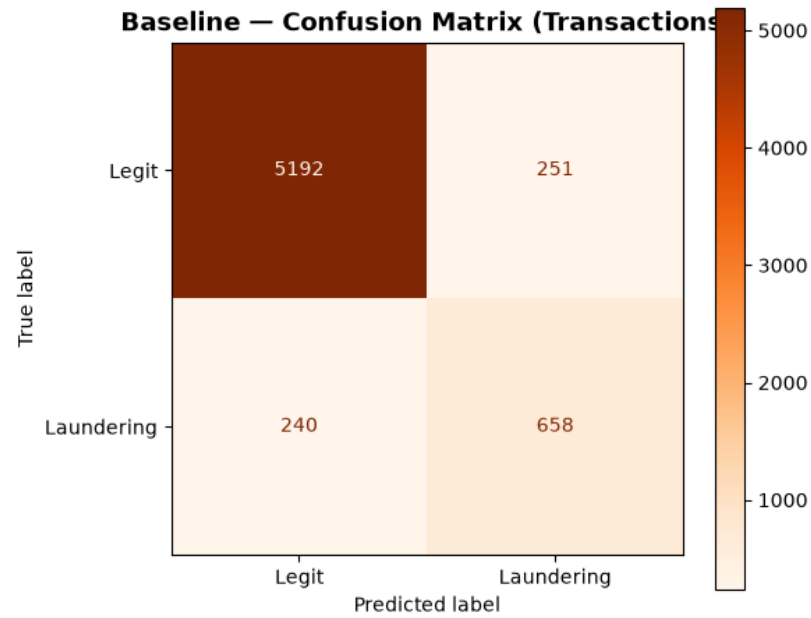


Figure 23: Confusion matrix for GAT baseline (Binary task)

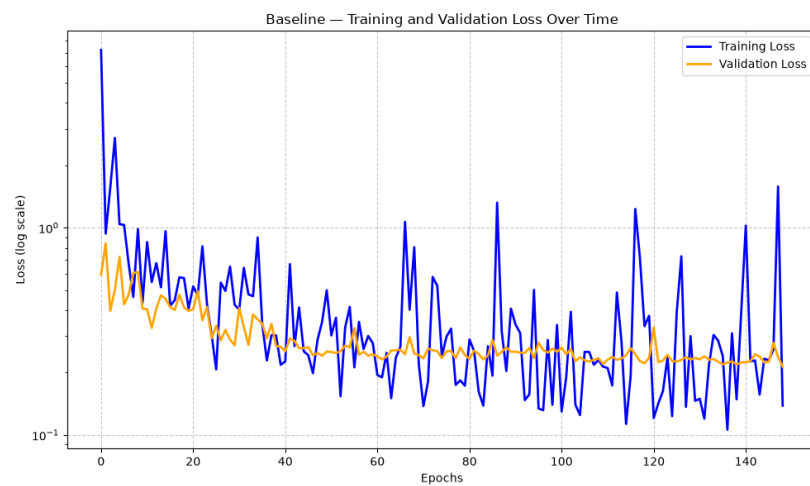


Figure 24: Training and patience loss for GIN baseline (Binary task)

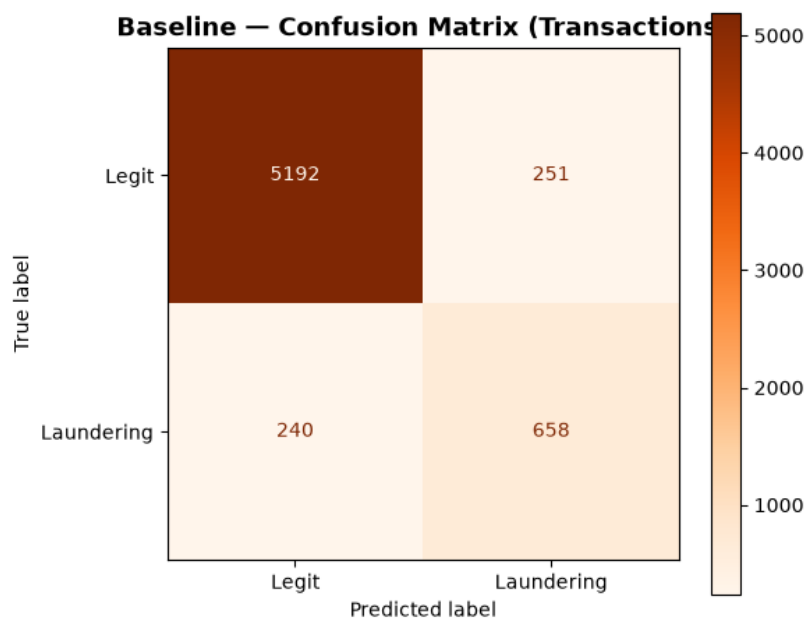


Figure 25: Confusion matrix for GIN baseline (Binary task)

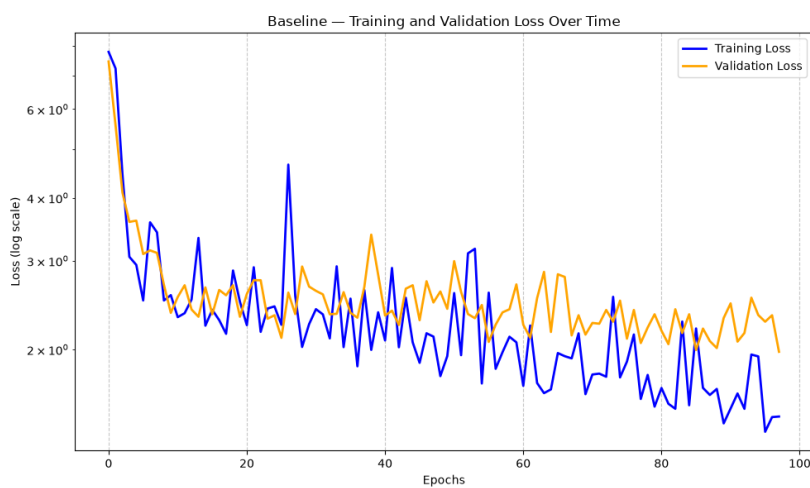


Figure 26: Training and patience loss for GAT baseline (Multiclass task)

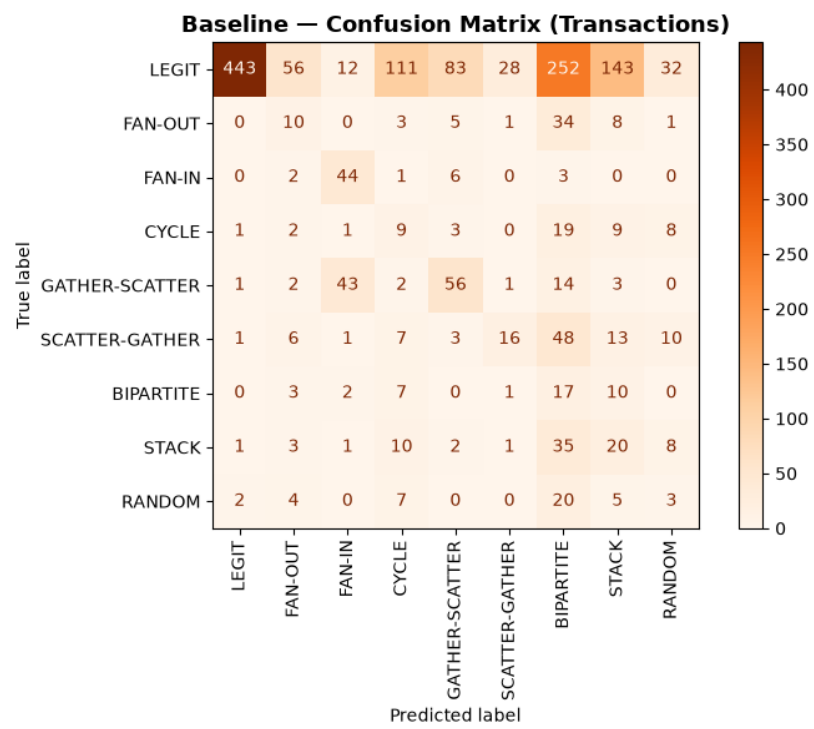


Figure 27: Confusion matrix for GAT baseline (Multiclass task)

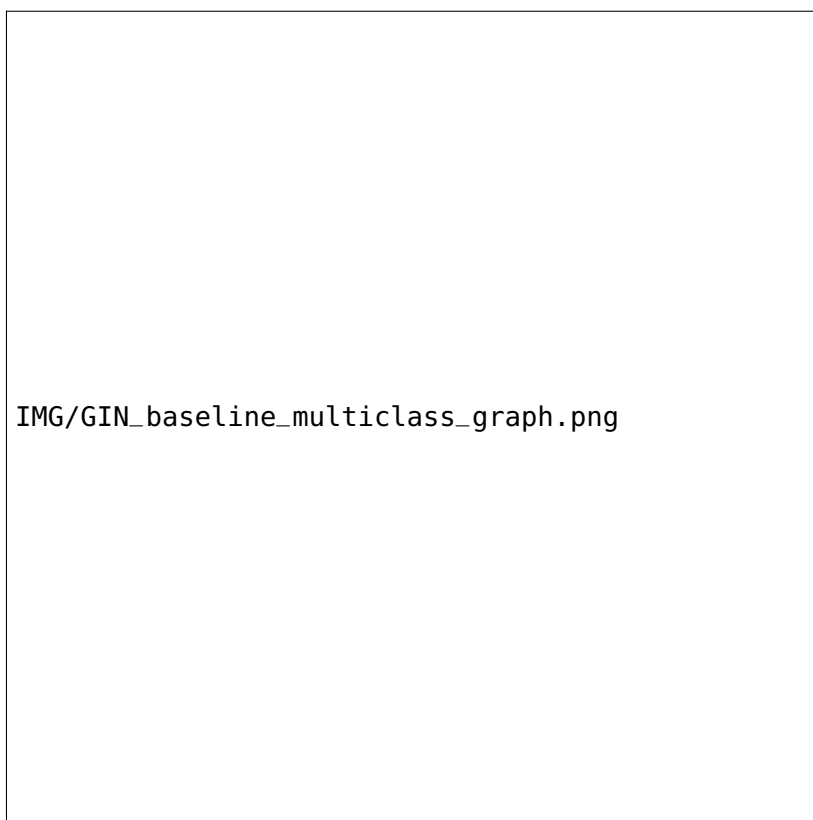


Figure 28: Training and patience loss for GIN baseline (Multiclass task)

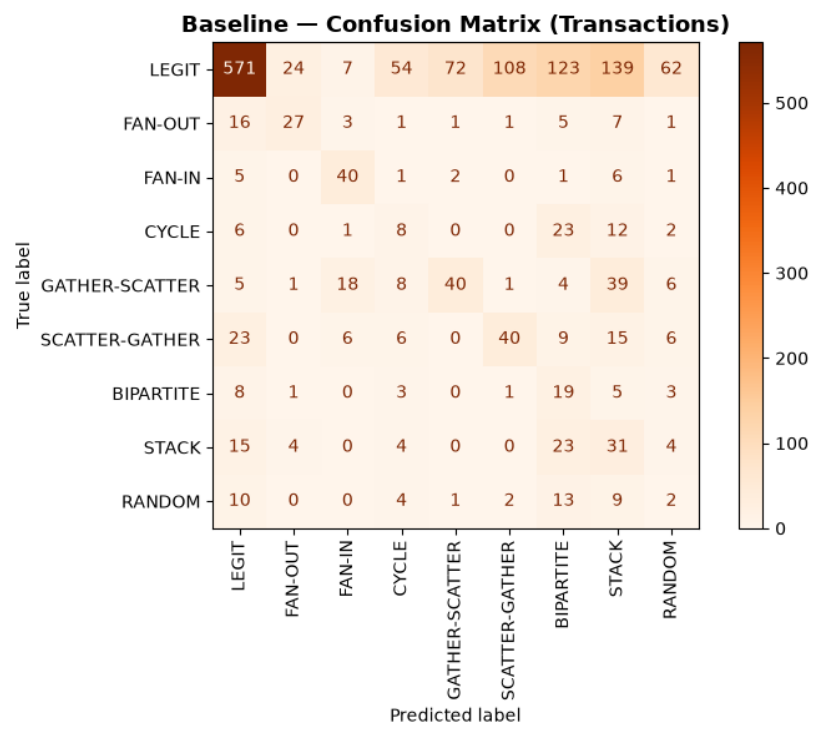


Figure 29: Confusion matrix for GIN baseline (Multiclass task)

CONCLUSIONS

5.1 FUTURE DIRECTIONS

A

SOFTWARE
